DLP Lab7: Policy-Based Reinforcement Learning

Yi-Hsiang Ho, 111550106

1. Introduction

Continuous control in high-dimensional action spaces presents significant challenges regarding sample efficiency, exploration, and training stability. In this report, I evaluate on-policy methods, specifically **Advantage Actor-Critic (A2C)** and **Proximal Policy Optimization (PPO)**, on the continuous tasks **Pendulum-v1** and **Walker2d-v5**.

To achieve stable policy convergence, my implementation highlights the critical role of specific stabilization techniques:

- **Gradient Clipping** (`clip_grad_norm_`): Essential for preventing extreme parameter shifts in both A2C and PPO, especially in bounding policy and value updates against gradient explosion.
- **Observation Normalization**: The single most critical continuous control trick for high-dimensional states (e.g., Walker2d-v5), normalizing state bounds dynamically to avoid gradient saturation.
- **Clipped Surrogate Objective & GAE**: Implemented in PPO to constrain policy update scales and balance return estimation bias and variance.

Additionally, I conducted ablation studies incorporating both a learning rate scheduler and advantage normalization in Task 3. The empirical results demonstrate that integrating these techniques alongside observation normalization yields significant synergistic improvements, achieving the highest returns and superior training stability compared to using observation normalization alone.

2. Implementation

2.1. Stochastic Policy Gradient and TD Error in A2C

Mathematical Concepts: Advantage Actor-Critic (A2C) is an on-policy method that optimizes a parameterized policy (the Actor) $\pi_\theta(a|s)$ using the stochastic policy gradient, while using a state-value function estimator (the Critic) $V_\phi(s)$ as a baseline to reduce gradient variance. The objective function $J(\theta)$ representing the expected return is maximized by updating policy parameters θ in the direction of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t,a_t\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) A(s_t,a_t)\right] \quad (1)$$

where $A(s_t,a_t)$ is the advantage function. In A2C, the advantage is estimated using the 1-step Temporal Difference (TD) error δ_t:

$$A(s_t,a_t) \approx \delta_t = Q(s_t,a_t) - V_\phi(s_t) \quad (2)$$

where the target action-value $Q(s_t,a_t)$ is computed from the 1-step empirical bootstrap:

$$Q(s_t,a_t) = r_t + \gamma V_\phi(s_{t+1})(1-d_t) \quad (3)$$

where γ is the discount factor and d_t is the environment termination/truncation boolean flag (`done`). The Critic parameters ϕ are optimized by minimizing MSE between the state-value prediction and the bootstrapped Q-target:

$$L_V(\phi) = \frac{1}{B}\sum_{i=1}^{B}\left(V_\phi(s_i) - Q_i^{\text{target}}\right)^2 \quad (4)$$

Code Implementation: In `a2c_pendulum-2.py` the Actor network outputs the mean (μ) and standard deviation ($\sigma = \exp(\text{log_std})$) of a Gaussian distribution. Continuous actions are sampled using PyTorch's `Normal` distribution class:

```python
class Actor(nn.Module):
    ...
    def forward(self, state: torch.Tensor) -> Tuple[torch.Tensor, Normal]:
        x = F.relu(self.hidden1(state))
        x = F.relu(self.hidden2(x))

        mu = torch.tanh(self.mu(x)) * 2.0
        log_std = torch.clamp(self.log_std(x), -20, 2)
        std = log_std.exp()

        dist = Normal(mu, std)
        action = dist.sample()

        return action, dist
```

During model updating in `update_model()`, we compute the bootstrapped target and critic value loss, and then update the critic network:

```python
class A2CAgent:
    ...
    def update_model(self) -> Tuple[torch.Tensor, torch.Tensor]:
        state, log_prob, next_state, reward, done = self.transition
        mask = 1 - done

        # Calcluate value loss (critic loss)
        next_state_t = torch.FloatTensor(next_state).to(self.device)
        next_value = self.critic(next_state_t)
        value = self.critic(state)

        q_value = reward + self.gamma * next_value * mask
        value_loss = F.mse_loss(value, q_value.detach())

        # update critic
        self.critic_optimizer.zero_grad()
        value_loss.backward()
        torch.nn.utils.clip_grad_norm_(self.critic.parameters(), 0.5)
        self.critic_optimizer.step()

        ...
```

The advantage δ_t is then computed by `q_value` and `value`, and the policy gradient is updated via backpropagation using the policy loss (negative objective) with entropy regularization:

```python
class A2CAgent:
    ...
    def update_model(self) -> Tuple[torch.Tensor, torch.Tensor]:
        ...
        value = self.critic(state)
        q_value = reward + self.gamma * next_value * mask
        ...

        # Calcluate policy loss (actor loss)
        advantage = (q_value - value).detach()
        _, dist = self.actor(state)
        entropy = dist.entropy().sum(dim=-1)

        policy_loss = -log_prob * advantage - self.entropy_weight * entropy
        policy_loss = policy_loss.mean()

        # update actor
        self.actor_optimizer.zero_grad()
        policy_loss.backward()
        torch.nn.utils.clip_grad_norm_(self.actor.parameters(), 0.5)
        self.actor_optimizer.step()

        ...
```

Both Actor and Critic networks undergo gradient norm clipping to 0.5 before stepping their respective Adam optimizers to maintain stability.

2.2. Clipped Surrogate Objective in PPO

Mathematical Concepts: Standard policy gradient algorithms are prone to destructively large policy updates. PPO resolves this by using a clipped surrogate objective that constrains the policy update step size. PPO defines the probability ratio $\rho_t(\theta)$ between the new and old policy:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{5}$$

The clipped surrogate objective $L_{\text{CLIP}}(\theta)$ is defined as:

$$L_{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{6}$$

where ϵ is the clipping parameter. The minimum operator takes the lower bound of the clipped and unclipped objective, which prevents the policy from driving the ratio ρ_t too far in a direction that exceedingly improves the objective, while allowing the objective to drop if the change is detrimental.

Code Implementation: In `ppo_pendulum-3.py` and `ppo_walker-3.py`, the PPO clipped surrogate objective is computed inside the mini-batch training loop over the rollout buffer. We evaluate the new probability density and compute the ratio and clipped actor loss as:

```python
class PPOAgent:
    ...
    def update_model(self, next_state: np.ndarray) -> Tuple[float, float]:
        ...
        for state, action, old_value, old_log_prob, return_, adv in ppo_iter:
            # calculate ratios
            _, dist = self.actor(state)
            log_prob = dist.log_prob(action)
            ratio = (log_prob - old_log_prob).exp()

            # actor_loss
            surr1 = ratio * adv
            surr2 = torch.clamp(ratio, 1.0 - self.epsilon, 1.0 + self.epsilon) * adv
            entropy = dist.entropy().sum(dim=-1, keepdim=True)
            actor_loss = -torch.min(surr1, surr2) - self.entropy_weight * entropy
            actor_loss = actor_loss.mean()
            ...
```

This objective is minimized (using the negative sign) to maximize the PPO objective while encouraging policy entropy.

2.3. Generalized Advantage Estimation (GAE)

Mathematical Concepts: GAE addresses the bias-variance trade-off in advantage estimation. GAE scales between the high-bias TD(0) advantage ($\tau = 0$) and the high-variance Monte Carlo advantage ($\tau = 1$) via an exponential discount factor $\tau \in [0, 1]$. The GAE advantage estimator $\hat{A}_t^{\text{GAE}}$ is given by:

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty}(\gamma\tau)^l \delta_{t+l}^V \tag{7}$$

where $\delta_t^V = r_t + \gamma V_\phi(s_{t+1})(1-d_t) - V_\phi(s_t)$ is the TD residual, and $(1-d_t)$ is the environment termination/truncation flag. The estimator is calculated recursively in reverse starting from the end of the trajectory:

$$\hat{A}_t^{\text{GAE}} = \delta_t^V + \gamma\tau(1-d_t)\hat{A}_{t+1}^{\text{GAE}} \tag{8}$$

The regression target for training the state-value function V_ϕ is then computed as the sum of the estimated advantage and the current state value:

$$R_t = \hat{A}_t^{\text{GAE}} + V_\phi(s_t) \tag{9}$$

Code Implementation: The recursive GAE is implemented in the `compute_gae()`. It starts from the end of the rollout buffers and moves backward in time:

```python
def compute_gae(
    next_value: list, rewards: list, masks: list, values: list, gamma: float, tau: float
) -> List:

    gae_returns = []
    gae = 0
    all_values = values + [next_value]

    for t in reversed(range(len(rewards))):
        delta = rewards[t] + gamma * all_values[t+1] * masks[t] - all_values[t]
        gae = delta + gamma * tau * masks[t] * gae
        gae_returns.insert(0, gae + all_values[t])

    return gae_returns
```

Here, `masks[t]` represents $(1 - d_t)$ to zero out the bootstrap when the episode terminates. The returned list contains the target returns R_t used to optimize the Critic's network parameters via MSE loss:

```
class PPOAgent:
    def update_model(self, next_state: np.ndarray) -> Tuple[float, float]:
        returns = compute_gae(
            next_value,
            self.rewards,
            self.masks,
            self.values,
            self.gamma,
            self.tau,
        )

        # Here return_ is return value from compute_gae
        for state, action, old_value, old_log_prob, return_, adv in ppo_iter(
            ...
            returns=returns,
            ...
        ):
            ...
            # critic_loss
            value = self.critic(state)
            critic_loss = F.mse_loss(value, return_)
            ...

```

2.4. Environment Sample Collection

A2C and PPO employ fundamentally different sample collection styles:

- **A2C (Online, Single-Step)**: Samples a single transition $[s_t, a_t, s_{t+1}, r_t, d_t]$, immediately updates both the Actor and Critic networks, and discards the transition.
- **PPO (Batch, Rollout Buffer)**: Collects a long trajectory of environment steps ($T = 2000$ or 2048) in buffers, recursively computes advantages/returns, and updates the models over multiple epochs via randomized mini-batches.

```
# --- A2C: Single-Step Collection ---
action = self.select_action(state)
next_state, reward, done = self.step(action)

actor_loss, critic_loss = self.update_model()

# --- PPO: Rollout Buffer Collection ---
for _ in range(self.rollout_len):
    action = self.select_action(state)
    next_state, reward, done = self.step(action)
    ...

actor_loss, critic_loss = self.update_model(next_state)
```

In PPO, `select_action()` and `step()` collect the trajectory. After collecting a full trajectory, `update_model()` will update the models multiple times via generator `ppo_iter()`.

2.5. Exploration Strategy in On-Policy RL

In this lab, since both A2C and PPO are on-policy algorithms, they can easily get stuck in local optima if the policy becomes deterministic too early. To ensure the agent continues to explore the continuous state-action space, two key mechanisms are introduced:

- **Stochastic Training vs. Deterministic Testing**: During training, the Actor network outputs the mean $\mu(s)$ and standard deviation $\sigma(s)$ of a Gaussian distribution. Actions are sampled from the distribution to encourage exploration of the environment. During testing (`self.is_test = True`), the agent deterministically selects the mean action (`dist.mean`) to ensure stable and optimal evaluation performance.
- **Entropy Penalty in Actor Loss**: To prevent the action standard deviation $\sigma(s)$ from collapsing to zero too quickly, an entropy regularization term is added to the Actor's loss function. Because the entropy of a Gaussian distribution increases with its variance, minimizing this negative term penalizes the network for shrinking standard deviations, successfully maintaining a healthy amount of exploration.

2.6. Weights & Biases (W&B) Model Performance Tracking

I integrated **Weights & Biases (W&B)** into our codebase to monitor model convergence, learning dynamics, and performance metrics in real-time. To ensure accurate comparative analysis, the x-axis for all logs is explicitly bound to the environment interaction step using `wandb.define_metric("step")` and aligning all logged parameters to it:

- **Actor Loss**: Shows the progress of policy optimization. In PPO, this reflects the clipped surrogate loss; in A2C, it shows the stochastic policy gradient loss.
- **Critic Loss**: Evaluates state-value estimation performance (MSE loss of $V(s)$ relative to bootstrapped/GAE returns).
- **Return / Eval Return**: Tracks the raw training return of each completed episode and the average evaluation score over 20 deterministic seed evaluations (run every `eval_interval` episodes).

2.7. Bonus: Continuous Control Optimization

Scaling PPO to high-dimensional continuous control locomotion like Walker2d-v5 is notoriously difficult due to large coordinate variances, complex joint dynamics, and sensitive reward signals. In Task 3 (`ppo_walker-3.py`), I incorporated several continuous control optimization strategies, which proved absolutely critical in enabling PPO to solve the environment:

Observation Normalization (`--obs-norm`): Raw state observations in Walker2d can have highly varying scales, which distort neural network weights and cause unstable gradient directions. I implemented observation normaliza-

tion using a running mean and standard deviation:

$$s_t^{\text{norm}} = \text{clip}\left(\frac{s_t - \mu_s}{\sqrt{\sigma_s^2 + 10^{-8}}}, -10, 10\right) \tag{10}$$

where μ_s and σ_s^2 are updated online using Welford's algorithm (`RunningMeanStd`). This normalizes all observation dimensions to zero-mean and unit variance.

```python
class RunningMeanStd:
    def __init__(self, shape):
        self.mean = np.zeros(shape, dtype=np.float64)
        self.var = np.ones(shape, dtype=np.float64)
        self.count = 1e-4

    def update(self, x):
        batch_mean = np.mean(x, axis=0)
        batch_var = np.var(x, axis=0)
        batch_count = x.shape[0]

        delta = batch_mean - self.mean
        tot_count = self.count + batch_count

        self.mean = self.mean + delta * batch_count / tot_count
        m_a = self.var * self.count
        m_b = batch_var * batch_count
        M2 = m_a + m_b + np.square(delta) * self.count * batch_count / tot_count
        self.var = M2 / tot_count
        self.count = tot_count
```

Advantage Normalization (`--adv-norm`): The advantage values computed over a rollout buffer can vary widely across iterations, causing huge fluctuations in actor updates. By normalizing the advantages across each mini-batch:

$$\hat{A}_t^{\text{norm}} = \frac{\hat{A}_t - \mu_A}{\sigma_A + 10^{-8}} \tag{11}$$

we force the advantage distribution to zero-mean and unit variance, keeping policy gradient updates exceptionally stable.

Learning Rate Cosine Scheduling (`--sched`): Using a fixed learning rate throughout training often leads to late-stage policy oscillations and prevents convergence to high-precision walking gaits. I integrated a Cosine Annealing learning rate schedule. Gradually annealing the learning rate allows PPO to make large, fast progress early on, and smoothly decay to precise, micro-level adjustments as the agent reaches the optimal gait.

3. Analysis and Discussions

3.1. Training Curves

The training curves represent the evaluation score, averaging across 20 test seeds, plotted against the total number of environment steps.

Task 1: A2C on Pendulum-v1. As shown in Fig. 1, A2C exhibits extremely high variance and slow convergence. Achieve around -150 at 120k steps, but quickly drop at 150k steps.

Task 2: PPO on Pendulum-v1. In Fig. 2, PPO exhibits more stable performance, reaching score -200 in just 30k steps.

Task 3: PPO on Walker2d-v5. In Fig. 3, Benefiting from continuous control optimizations, Walker2d-v5 achieves steady growth, surpassing an average return of 2500 within 1M steps. And finally Achieves 4000 score around 2M steps.



Figure 1. **Task 1 A2C Pendulum-v1** training curve.

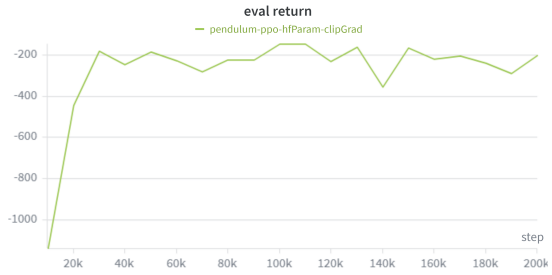


Figure 2. **Task 2 PPO Pendulum-v1** training curve.



Figure 3. **Task 3 PPO Walker2d-v5** training curve.

3.2. Evaluation Results

The screenshots of evaluation results are in Fig. 4, Fig. 5, and Fig. 6. **Task 1 A2C Pendulum-v1** achieve -143.4147 for its best checkpoint saved in 19.8k steps, while **Task 2 PPO Pendulum-v1** achieve -145.9427 in 10k steps. Both Task successfully achieve score above -150. The scores for **Task 3 PPO Walker2d-v5** are summarized in Tab. 1. Within 1M steps, the model has already performed over 2500 score.

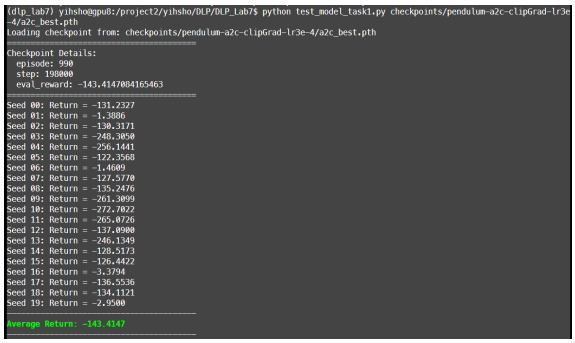


Figure 4. **A2C Pendulum-v1** evaluation terminal output. The checkpoint is selected by highest evaluated score.

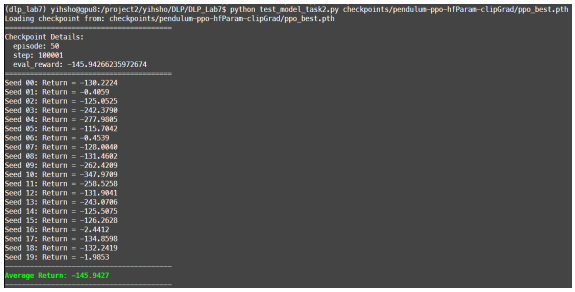


Figure 5. **PPO Pendulum-v1** evaluation terminal output. The checkpoint is selected by highest evaluated score.

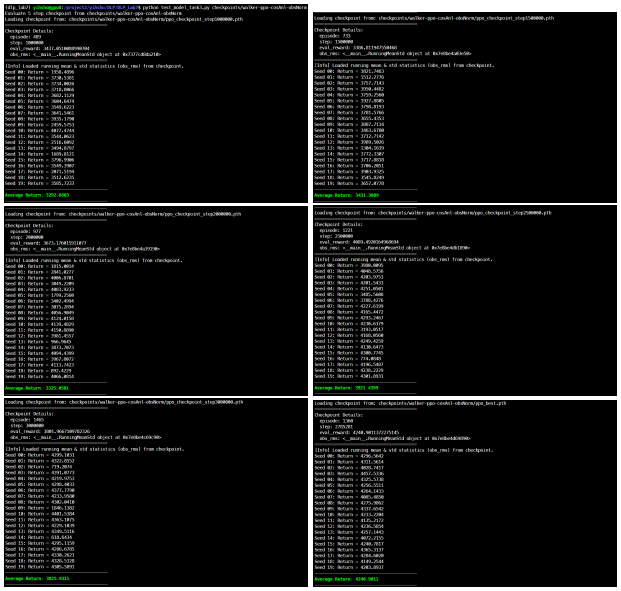


Figure 6. **PPO Walker2d-v5** evaluation terminal output. The checkpoints are stored at step 1M, 1.5M, 2M, 2.5M, 3M, and best.

Method	1M	1.5M	2M	2.5M	3M	best
PPO	3292	3431	3325	3921	3825	4240

Table 1. **PPO Walker2d-v5** average return in different training steps. It is tested with 20 random seed for each step.

3.3. Comparison of Sample Efficiency and Training Stability

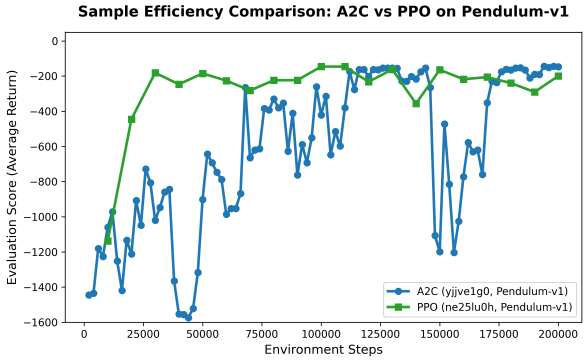


Figure 7. **Sample Efficiency Comparison** of A2C vs. PPO.

Sample Efficiency: PPO is immensely more sample-efficient than A2C. A2C is a purely single-step method (TD(0)) that computes gradient updates and immediately discards transition samples. PPO, however, utilizes a rollout buffer that stores transitions. It then performs multiple epochs (`update_epoch = 10`) of optimization on these stored transitions using mini-batch gradient descent. Furthermore, PPO's GAE advantage estimator ($\tau = 0.95$) provides a much lower-variance target compared to A2C's high-variance single-step TD error, allowing the policy to learn much faster per environment sample. As illustrated in Fig. 7, both agents achieve stable convergence within 200k steps, with PPO showing faster initial policy optimization but A2C ultimately reaching a competitive evaluation return under robust exploration.

Training Stability: A2C suffers from high training instability and is highly susceptible to policy collapse. Because A2C's updates do not restrict the policy update size, a single outlier gradient step can severely distort the policy, causing it to become highly deterministic in a suboptimal state. This is reflected in A2C's highly volatile training curves with frequent performance drops in Fig. 7. Conversely, PPO-Clip guarantees stable, monotonic policy improvements because its clipped surrogate objective bounds the ratio $r_t(\theta)$ within $[1-\epsilon, 1+\epsilon]$. This prevents catastrophic policy changes, maintaining a smooth and stable training trajectory.

3.4. Empirical Parameter Study

I conducted ablation studies on PPO's clipping parameter ϵ and the policy entropy coefficient c_e on the high-dimensional Walker2d-v5 environment over 3 million environment steps. For all runs, the all continuous control tricks (observation normalization, advantage normalization, and cosine learning rate scheduling) were enabled to isolate the impact of the policy optimization boundaries and exploration weights.

Clipping Parameter (ϵ): The clipping parameter ϵ dictates the boundary of policy updates. I evaluated $\epsilon \in \{0.05, 0.20, 0.50, 1.00\}$, with the evaluation results summarized in Tab. 2. Bounding updates too tightly ($\epsilon = 0.05$) ensures high stability but slows down early-stage learning, although it ultimately converges to a highly competitive score. The optimal setting ($\epsilon = 0.20$) balances stability and convergence speed, yielding the fastest learning progress. Relaxing the constraint further ($\epsilon = 0.50$ and 1.00) severely degrades performance, leading to high training instability, early policy collapse, and significantly lower returns.

Clipping (ϵ)	**1M**	**1.5M**	**2M**	**2.5M**	**3M**	**best**
0.05	910	1739	**3545**	**4107**	**4143**	4313
0.20	**2924**	**2979**	2791	3978	3901	**4367**
0.50	1754	2171	2451	3099	3274	3414
1.00	293	540	627	2451	2033	2472

Table 2. **Clipping parameter ablation** average evaluation return at different training steps on Walker2d-v5. The best result in each time steps is marked inn bold.

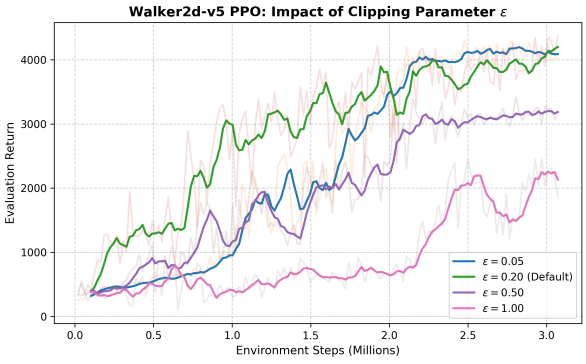


Figure 8. **Clipping parameter ablation**. Solid curves represent the smoothed evaluation returns using a simple moving average with a window size of 5, while the raw unsmoothed curves are plotted faintly in the background. Bounding updates too tightly ($\epsilon = 0.05$) limits initial learning progress, while excessively loose boundaries ($\epsilon = 1.00$) lead to highly suboptimal behaviors.

Entropy Coefficient (c_e): The entropy coefficient c_e regulates the exploration-exploitation trade-off. I evaluated $c_e \in \{0.00, 0.001, 0.01, 0.10\}$, with the performance metrics documented in Tab. 3. Without sufficient entropy regularization ($c_e = 0.00$ and 0.001), the policy quickly collapses to a near-deterministic state and gets trapped in suboptimal gaits. While the moderate value ($c_e = 0.01$) enables smooth gait learning, the high entropy coefficient ($c_e = 0.10$) delivers the absolute best performance. This demonstrates that high-dimensional locomotion environments benefit significantly from persistent exploration pressure to discover highly efficient walking gaits.

Entropy (c_e)	**1M**	**1.5M**	**2M**	**2.5M**	**3M**	**best**
0.00	2676	2678	3203	4025	3752	4151
0.001	1896	2214	2640	3346	3758	3956
0.01	2924	2979	2791	3978	3901	4367
0.10	**4136**	**4385**	**4924**	**5490**	**5345**	**5693**

Table 3. **Entropy coefficient ablation** average evaluation return at different training steps on Walker2d-v5. The best result in each time steps is marked in bold.

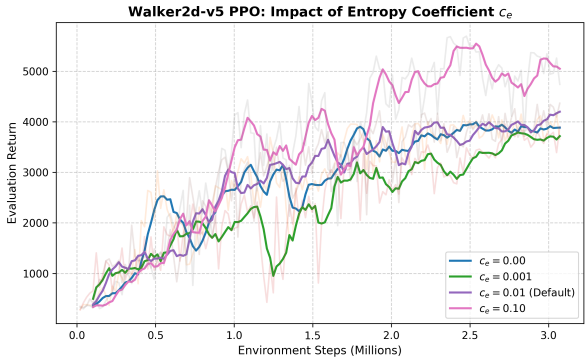


Figure 9. **Entropy coefficient ablation**. Solid curves represent the smoothed evaluation returns using a simple moving average with a window size of 5, while raw curves are plotted faintly in the background. A higher entropy weight ($c_e = 0.10$) promotes robust exploration, yielding a substantial performance boost in the complex Walker2d locomotion task, whereas too little entropy leads to premature convergence in local optima.

4. Additional Analysis: Ablation Study on Continuous Control Optimizations

To analyze the impact of individual continuous control optimization strategies, I conducted comprehensive ablation experiments on the Walker2d-v5 environment. The first study (Fig. 10) investigates the effect of adding each trick individually to Vanilla PPO, while the second study

(Fig. 11) examines the performance degradation when individual tricks are subtracted from the full configuration.

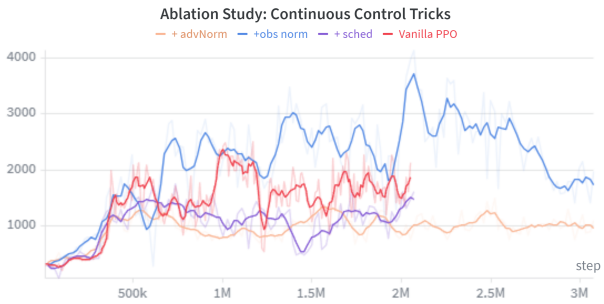


Figure 10. **Adding individual continuous control tricks to Vanilla PPO**.

The "Adding" study (Fig. 10) reveals that Observation Normalization (`+obs norm`) is the single most critical individual factor for successful policy optimization. Adding only observation normalization enables PPO to achieve a rapid, substantial performance surge, peaking near a return of 3700 around 2.1M steps. This is because standardizing the high-dimensional Walker2d observations to zero-mean and unit variance prevents gradient saturation in early layers. In contrast, adding only learning rate scheduling (`+sched`) or only advantage normalization (`+advNorm`) in isolation fails to overcome the state-scaling bottleneck, resulting in worse performance than vanilla PPO. However, the solitary `+obs norm` run also highlights its limitations: without learning rate annealing or mini-batch stabilization, the policy suffers from late-stage training instability, and decays back to approximately 1800 by 3M steps.

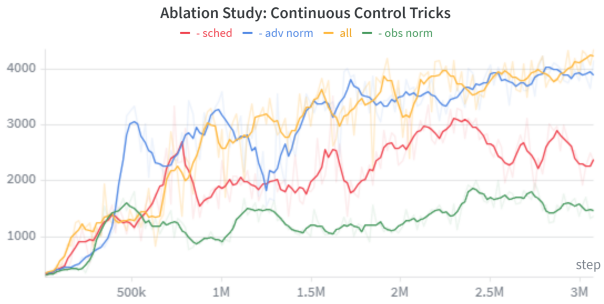


Figure 11. **Subtracting individual continuous control tricks from the full configuration**.

The "Subtracting" study (Fig. 11) confirms the synergistic necessity of all three optimizations. The full configuration (`all`), combining all three tricks, achieves the highest return with highly stable, monotonic growth. In contrast, removing observation normalization (`-obs norm`) causes a catastrophic drop, pinning performance below 1800 and demonstrating its role as the absolute cornerstone. Removing the learning rate scheduler (`-sched`) allows efficient early learning but results in a mid-stage peak near 3100 followed by a steady decay to 2400, illustrating the necessity of learning rate annealing to prevent late-stage policy degradation. Lastly, while removing advantage normalization (`-adv norm`) yields rapid early acceleration (reaching 3000 at 500k steps) and a competitive final return of 3900, it exhibits larger training variance and slightly lower asymptotic performance, highlighting its vital role in mini-batch stabilization.

5. Conclusion

In this lab, I successfully implemented and analyzed policy-based reinforcement learning using A2C and PPO. The experimental evaluations on Pendulum-v1 highlight that PPO-Clip offers significantly higher sample efficiency and exceptional training stability compared to A2C, which is highly prone to gradient volatility and suboptimal policy convergence. Furthermore, scaling PPO to the high-dimensional Walker2d-v5 environment demonstrates that observation normalization, advantage normalization, and cosine learning rate scheduling are indispensable continuous control strategies. This lab successfully highlights the importance of matching robust loss formulation with proper state conditioning and stabilization tricks to achieve state-of-the-art continuous control performance.